System Design Decisions & Trade-offs

1. System Objective and Architecture

The Banking Loan Risk Analysis Application processes the UCI Default of Credit Card Clients dataset to support portfolio exposure analysis, repayment trends, customer risk review, and analyst reporting.

The application follows a layered architecture:

Input CSV → Python/Pandas ETL and Validation → MySQL Relational Database → Risk Scores and Reporting Aggregates → FastAPI REST APIs → Streamlit User Interface

ETL logs, record-level audit tables, and data-quality issue tracking provide traceability throughout the data-processing workflow.

2. Design Decisions and Associated Trade-offs

2.1 Modular, File-Based Python ETL

Design Decision: The ETL pipeline is divided into separate modules: extract.py, data_quality.py, transform.py, incremental.py, load.py, reconciliation.py, and pipeline.py. Input files are managed through incoming, processed, rejected, and archive directories.

Benefits: Modular design improves code maintainability, testing, debugging, and reusability. File-based processing also simplifies local execution.

Trade-offs: File-based ingestion requires careful file movement and operational control. It is not designed for distributed real-time streaming.

2.2 Data Validation Before Database Loading

Design Decision: Data-quality checks validate incoming records before loading them into curated database tables. Invalid records can be rejected, exported, and tracked for investigation.

Benefits: Prevents invalid records from affecting financial calculations, customer risk scores, and reporting accuracy. Improves data reliability and auditability.

Trade-offs: Strict validation may reject partially usable records. Additional validation and exception handling increase ETL processing complexity.

2.3 Hash-Based Incremental ETL

Design Decision: The incremental.py module compares incoming customer records with existing database records using stored record_hash values.

Benefits: Avoids unnecessary database updates, reduces redundant processing, and supports repeated ingestion of overlapping datasets.

Trade-offs: Hash-generation logic and customer identification rules must remain consistent. Database lookups introduce additional processing overhead.

2.4 ETL Run Tracking and Record-Level Auditing

Design Decision: The database includes three ETL monitoring tables:

- etl_runs – Tracks ETL execution details.
- etl_record_audit – Records processing information at the individual record level.
- data_quality_issues – Stores identified validation issues.

The pipeline generates unique run identifiers and performs reconciliation checks.

Benefits: Supports data lineage, troubleshooting, execution monitoring, and accountability. Failed or rejected records can be investigated.

Trade-offs: Maintaining detailed audit records increases database storage requirements and write operations. Audit retention policies may be needed as data volumes increase.

2.5 Normalized MySQL Database Design

Design Decision: MySQL is used as the relational database, with SQLAlchemy managing database connectivity.

The core data is separated into four relational tables: customers, credit_profiles, bills, and payments.

Benefits: Reduces data duplication, improves consistency, supports relational integrity, and allows customer, credit, billing, and payment information to be queried independently.

Trade-offs: Complex analytical queries require joins across multiple tables. Database indexing and schema maintenance become important as the dataset grows.

2.6 Rule-Based Credit Risk Scoring

Design Decision: A deterministic rule-based risk engine calculates customer risk indicators using configurable thresholds and weights defined in risk_config.py.

The system classifies customers into LOW, MEDIUM, and HIGH risk bands, with an UNCLASSIFIED category for applicable missing-value conditions. Risk calculations are implemented in risk_engine.py.

Benefits: Risk assessments are transparent, explainable, configurable, and do not require machine-learning model training.

Trade-offs: Rule-based scores depend on predefined thresholds and may not capture complex relationships between financial variables. The scores are risk indicators rather than statistically calibrated default probabilities.

2.7 Separate Risk Assessment and Review Tables

Design Decision: Risk assessments and manual review information are maintained in separate tables:

- customer_risk_scores – Stores calculated customer risk assessments.
- customer_risk_reviews – Stores customer risk review information.

Benefits: Separates automated risk calculations from analyst review activity. Supports traceability and structured risk management.

Trade-offs: Risk scores and review statuses must be coordinated to prevent inconsistencies between automated assessments and human decisions.

2.8 Precomputed Reporting Aggregate Tables

Design Decision: Dedicated reporting tables store frequently used analytical summaries:

- agg_portfolio_summary
- agg_risk_distribution
- agg_monthly_financials
- agg_segment_risk

The refresh_aggregates.py module refreshes these summaries using transactional database operations.

Benefits: Improves dashboard performance by reducing repeated calculations over detailed transactional records. Transactional refreshes help maintain consistency.

Trade-offs: Aggregate tables may contain outdated information until refreshed. Full refresh operations can become expensive as data volume increases.

2.9 FastAPI for REST API Development

Design Decision: FastAPI is used to expose application functionality through REST endpoints. The backend includes dashboard, customer, export, review, and authentication routers.

Benefits: Provides a modular service layer, supports request and response validation, enables API testing, and separates backend business logic from frontend presentation.

Trade-offs: Introducing an API layer increases application complexity and requires endpoint maintenance, error handling, and coordination between frontend and backend components.

2.10 Streamlit-Based User Interface

Design Decision: Streamlit is used to build the application's interactive frontend.

The frontend includes app.py, dashboard.py, customers.py, reviews.py, and exports.py.

Benefits: Enables rapid development of dashboards, customer exploration, risk reviews, and data-export features using Python.

Trade-offs: Streamlit provides less flexibility for highly customized client-side interactions compared with frontend frameworks such as React.

2.11 JWT-Based Authentication and Role-Based Authorization

Design Decision: The backend implements JWT authentication using HS256 and distinguishes between analyst and administrator roles.

JWT validation is implemented in security.py, with a configured token lifetime of 30 minutes.

Benefits: Supports stateless authentication, separates user permissions, and protects API operations through role-based authorization.

Trade-offs: Secure token management, secret rotation, token revocation, and session handling require additional operational controls. JWT authentication alone does not provide a complete identity-management solution.

2.12 Automated Testing and Logging

Design Decision: Automated tests cover ETL processing, data validation, incremental loading, reconciliation, risk rules, APIs, reporting aggregates, and frontend integration.

The repository includes a saved test report showing 293 passing tests.

Benefits: Helps identify regressions, improves maintainability, and supports validation of important application workflows. Logging assists in identifying ETL execution failures.

Trade-offs: Automated tests require ongoing maintenance as application requirements change. Previously passing tests do not guarantee correctness in every deployment environment.

3. Data Persistence and Auditability

The database architecture separates operational data, risk assessments, ETL monitoring, and reporting summaries.

Category	Database Tables
Core operational data	customers, credit_profiles, bills, payments
Risk management	customer_risk_scores, customer_risk_reviews

ETL monitoring	etl_runs, etl_record_audit, data_quality_issues
Reporting aggregates	agg_portfolio_summary, agg_risk_distribution, agg_monthly_financials, agg_segment_risk

This separation improves maintainability and supports independent management of customer information, generated risk assessments, processing history, and dashboard reporting.

Record-level auditing and reconciliation also help identify inconsistencies between source records and loaded database records.

4. Reliability, Security, and Performance Considerations

Reliability: The system uses data validation, rejected-record handling, unique ETL run identifiers, record-level reconciliation, and transactional aggregate refreshes to reduce the risk of inconsistent data.

Security: JWT authentication and role-based authorization protect backend API operations. Authentication secrets are obtained from environment configuration rather than being embedded directly in application logic.

Performance: Hash-based incremental processing reduces unnecessary database updates. Chunked customer lookups, SQL indexes, and precomputed reporting aggregates improve data-processing and query efficiency.

These mechanisms improve application reliability and maintainability but introduce additional storage, configuration, and operational requirements.

5. Limitations and Recommended Future Improvements

The application is designed around a Python, MySQL, FastAPI, and Streamlit technology stack suitable for local execution and demonstration.

Although the system implements modular ETL processing, risk classification, reporting, and API-based access, several improvements could enhance its scalability and operational capabilities.

Recommended Future Improvements:

1. Automated ETL Scheduling: Introduce an orchestration tool such as Apache Airflow for scheduled ingestion, monitoring, and failure notifications.

2. Database Migration Management: Implement version-controlled database migrations to manage future schema changes.

3. Enhanced Authentication: Add stronger identity management, token revocation, and secret-rotation mechanisms.

4. Audit Data Retention: Define retention and archival policies for ETL execution logs and record-level audit tables.

5. Incremental Aggregate Refresh: Replace full aggregate refreshes with incremental updates where appropriate to improve performance.

6. Risk Model Validation: Evaluate and refine risk-scoring thresholds using historical repayment outcomes and relevant financial risk metrics.

7. Performance Optimization: Conduct database load testing, query benchmarking, and API response-time analysis.

8. Deployment Improvements: Introduce containerization, automated deployment, and centralized monitoring for more reliable deployment environments.

These enhancements would improve scalability, maintainability, and operational readiness while preserving the existing modular architecture.